

nopCommerce Federated Commerce

Two independent online stores — shared event backbone, unified product search, and single sign-on

2

Independent Stores

~5 s

Save → Searchable

6

Custom Features

14

Dashboards

Event

Outbox Plugin

Product changes are captured in the database as events — guaranteed, no data loss.

Relay

Kafka Relay

Events are forwarded to the shared Kafka message bus automatically every 5 s.

Index

Meilisearch Indexer

Products from both stores land in one searchable index in real time.

Search

Discovery API + Widget

Shoppers search both stores from a single search bar.

Identity

Keycloak SSO

One account works on both stores — no second registration.

Ops

Prometheus + Grafana

14 dashboards track the full pipeline end-to-end.

Features Built on Top of nopCommerce

Outbox Plugin

Every product change is written to the database as an event in the same transaction — guaranteeing it is never lost even if downstream services are down.

Kafka Relay

A background worker polls the outbox every 5 seconds and forwards events to Kafka, with automatic retry, dead-message handling, and a self-healing watchdog.

Meilisearch Indexer

Consumes Kafka events in batches and upserts or removes products from the shared search index, keeping both stores searchable in near real-time.

Discovery API + Widget

A lightweight API exposes unified cross-BU search and facets; an embedded JS widget lets shoppers search both stores from any page without leaving their current store.

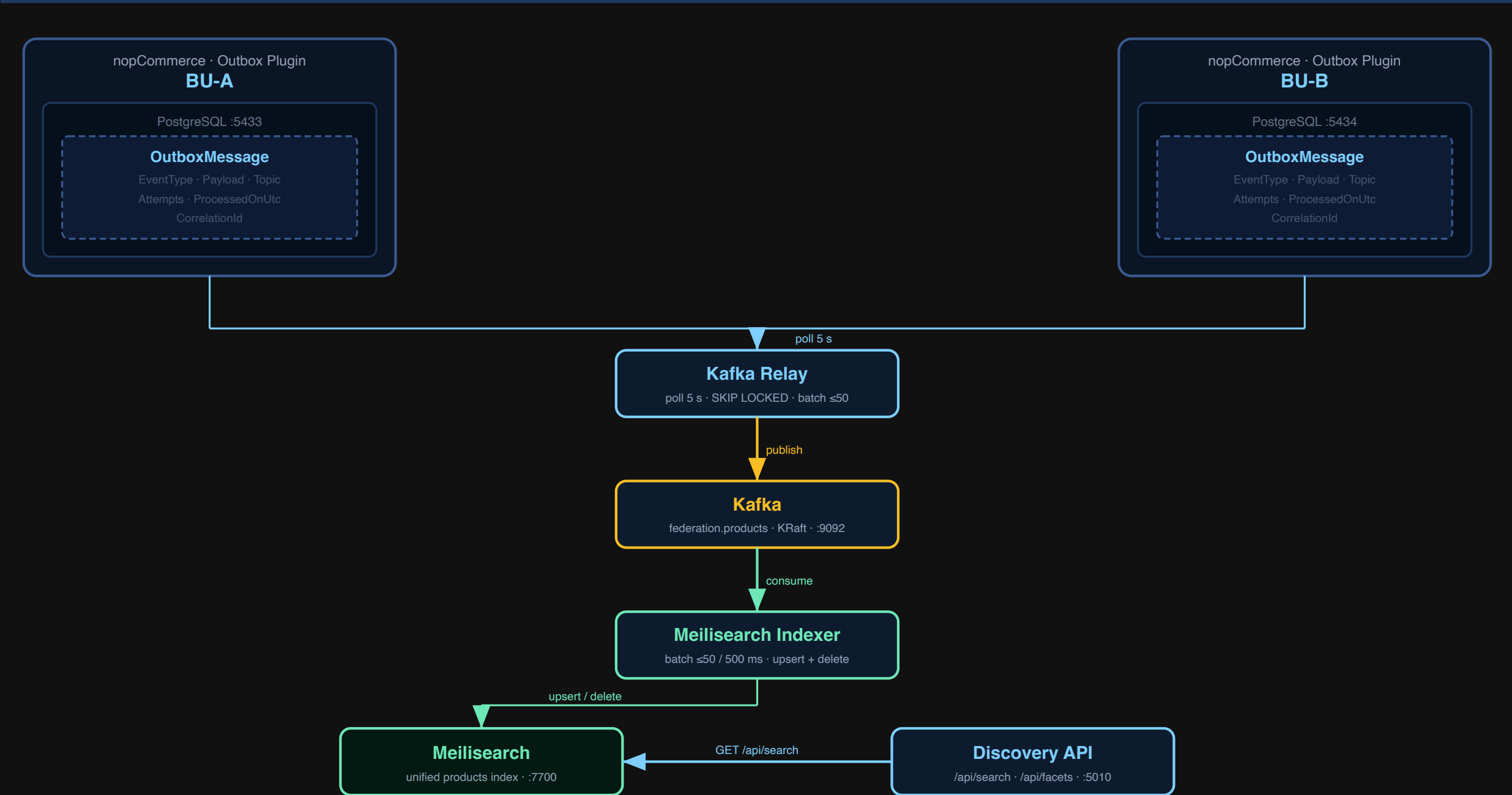
Keycloak SSO

A custom nopCommerce external-auth plugin integrates Keycloak OIDC so one account grants access to both stores with no extra registration.

Prometheus + Grafana

14 dashboards cover every pipeline stage — from outbox pending counts and dead messages to cross-BU search latency, SSO health, and side-by-side BU comparison.

Pipeline Architecture



Step by Step — Admin Publishes a Product

- 1 Admin saves a product** in the nopCommerce admin panel
nopCommerce fires an internal event: `EntityInsertedEvent<Product>`
- 2 The Outbox Plugin writes a row** to the `OutboxMessage` database table — in the *same transaction* as the product save
Row contains: store, product id, name, URL, categories, price, event type
- 3 The Kafka Relay polls every 5 seconds**, picks up the row, and sends it to the `federation.products` Kafka topic
Once Kafka confirms delivery, the row is marked as processed
- 4 The Meilisearch Indexer reads from Kafka** and upserts the product into the shared search index
Document id = `bu-a-42` (store + product id) — unique across both stores
- 5 The product is now searchable** — Discovery API returns it for both stores · **Total time: ~5–6 s**
`GET /api/search?q=armchair` → hits from BU-A and BU-B in one response

The Outbox Pattern — Events That Are Never Lost

Why not write directly to Kafka?

If Kafka was temporarily down when a product was saved, the product would be saved but the event would be lost — search would never know. The outbox writes the event to the **database in the same transaction**, so both are saved or neither is.

What if delivery fails?

If the relay cannot reach Kafka, the attempt counter increases. The row is retried on the next cycle. After **5 failures** it becomes a *Dead Message* — skipped so other events are not blocked. It can be re-queued manually at any time.

What is stored

Field	Example
Event type	product.published
Store	bu-a
Payload	name, URL, price, categories...
Status	Pending → Processed
Attempts	0 (increments on failure)

Delete events

When a product is deleted, only the **store code and product ID** are stored — that is all the indexer needs to remove it from search. A second listener on the admin activity log acts as a fallback to catch any missed deletes.

Idempotency

Each row has a deterministic ID based on store + event type + product id. The same event can be safely re-delivered without creating duplicates in the search index.

Relay → Kafka → Meilisearch — How Events Travel

The Relay Worker (every 5 s)

```
SELECT * FROM "OutboxMessage"
WHERE ProcessedOnUtc IS NULL
      AND Attempts < 5
ORDER BY CreatedOnUtc
LIMIT 50
FOR UPDATE SKIP LOCKED
```

SKIP LOCKED — two relay workers can run at once without processing the same row twice. **Acks=All** — no silent Kafka data loss.

Self-Healing Watchdog (every 5 min)

If the Meilisearch index is ever empty — e.g. after a server restart wipes the volume — the watchdog detects it and re-queues all processed product events. The index rebuilds itself in **~30 seconds with no manual action**.

The Indexer — Batch Processing

Rather than updating Meilisearch once per product, the indexer waits up to **500 ms** and collects up to **50 messages**, then sends one bulk request. This reduces 50 HTTP calls to just 2.

Event → Index action

Event	Action
product.published	Add / update in index
product.updated	Add / update in index
product.unpublished	Remove from index
product.deleted	Remove from index

Deletes wait for Meilisearch to confirm the task completed before moving on.

Discovery — How Shoppers Search Both Stores

The Unified Index

Products from both stores live in one Meilisearch index. Each document carries the store it came from.

```
{
  "id":      "bu-a-42",
  "storeName": "HomeStyle Living",
  "productName": "Armchair Velvet",
  "productUrl": "http://localhost:5001/armchair-velvet",
  "categories": ["Furniture", "Living Room"],
  "price":    349.00
}
```

Discovery API (:5010)

```
GET /api/search?q=armchair
GET /api/facets
```

Results cached for **1 second**. A deleted product disappears within 1 s of the indexer confirming removal.

Embedded Search Widget

A JavaScript widget is included on each store's homepage. Shoppers search both stores without leaving the page. Results include a direct "View on store" link.

Discovery Web Portal (:5011)

A standalone cross-BU search portal with store filter chips — narrow results to BU-A, BU-B, or both.

End-to-end timing

Plugin writes row	<10 ms
Relay → Kafka	≤5 s
Indexer → Meilisearch	≤500 ms
Cache expires	1 s
Total	~5–6 s

Single Sign-On — One Account for Both Stores

Keycloak is the central identity provider. A shopper creates one account and can log in to both stores — no second registration, no second password.

- 1 Shopper clicks "Login" on BU-A or BU-B
- 2 Browser → Keycloak :8080 — the central login page
`/realms/nop-federation/.../auth?client_id=bu-a-client`
- 3 Shopper enters credentials once — Keycloak validates
- 4 Keycloak redirects back to the store with a signed identity token
- 5 The plugin maps the token to a nopCommerce customer record
✔ Session established — same account works on BU-B too

Two-Route Login Design

The login uses two URL routes so the security token validation and the account mapping never interfere:

- `/callback` — OIDC middleware validates the token
- `/login-callback` — plugin controller maps identity to Customer

What this means for shoppers

A customer who registered on BU-A can log in to BU-B immediately. Their name and identity carry over.
No extra steps.

Config

Realm: `nop-federation`
Clients: `bu-a-client`, `bu-b-client`
Exported: `FederationPlatform/keycloak-realm.json` — loads on startup

Observability — Knowing the Pipeline is Healthy

Key metrics (auto-collected)	
Metric	Tells you
Outbox pending	Events waiting to be relayed
Dead messages	Events that failed 5 times
Oldest pending age	How far behind the relay is
Index doc count	Products in the search index
Service health	Is every container up?

Quick diagnostic commands

```
./platform.sh outbox bu-a 10
docker logs federation_kafka_relay --tail 50
curl http://localhost:7700/indexes/products/stats
curl "http://localhost:5010/api/search?q=chair"
```

Infrastructure

- Pipeline E2E — all 5 stages at a glance
- Outbox — pending, dead messages, event types
- Relay — throughput, per-store health
- Indexer — write rate, search rate, coverage
- Latency SLA — p50/p95/p99, compliance %
- Search — success rate, top queries
- SSO — login rate, sessions, response times

Business

- Business Pulse — product, order & customer volumes
- Demand Channels — search rate per store
- Resilience & SLA — availability, backlog
- **BU Comparison** — BU-A vs BU-B on every KPI

Grafana: <http://localhost:3000> · admin / admin

Summary



Built on nopCommerce

Core platform never modified. All features are plugins and standalone services — upgradeable at any time.



Events Never Get Lost

Outbox pattern guarantees delivery even if Kafka or the indexer is temporarily down.



~5 s End-to-End

Admin save → visible in cross-BU search. Fully automatic, zero manual steps.



Unified Search

One search bar, both stores. Widget on each homepage + standalone Discovery Web portal.



Self-Healing

Watchdog detects empty index → auto-replays outbox → full index rebuilt in ~30 s.



Full Visibility

14 Grafana dashboards. Every stage tracked from event write to shopper search result.

```
docker compose -f docker-compose.bua.yml -f docker-compose.bub.yml -f docker-compose.federation.yml up -d
```